Coração Sob Controle

Relatório técnico do MVP integrado

Grupo São Paulo e Interior | Tecnologia em Inteligência Artificial | 31/10/2026

A fase 7 transforma as entregas anteriores em uma aplicação com login, leitura de sinais, inferência e visualização Web/Android. O escopo prioriza um único backend Python, duas interfaces e um contrato simples de integração com MicroPython, sem infraestrutura distribuída adicional.

Objetivo e escopo

Demonstrar o fluxo Sensor → MicroPython → Backend → IA → UI com dados simulados, disponibilizar a Web na Vercel e gerar APK via Expo EAS. O núcleo já conecta modelos de risco e linguagem, visão computacional e histórico de leituras. A captura física/simulada no Wokwi é responsabilidade de Raphael.

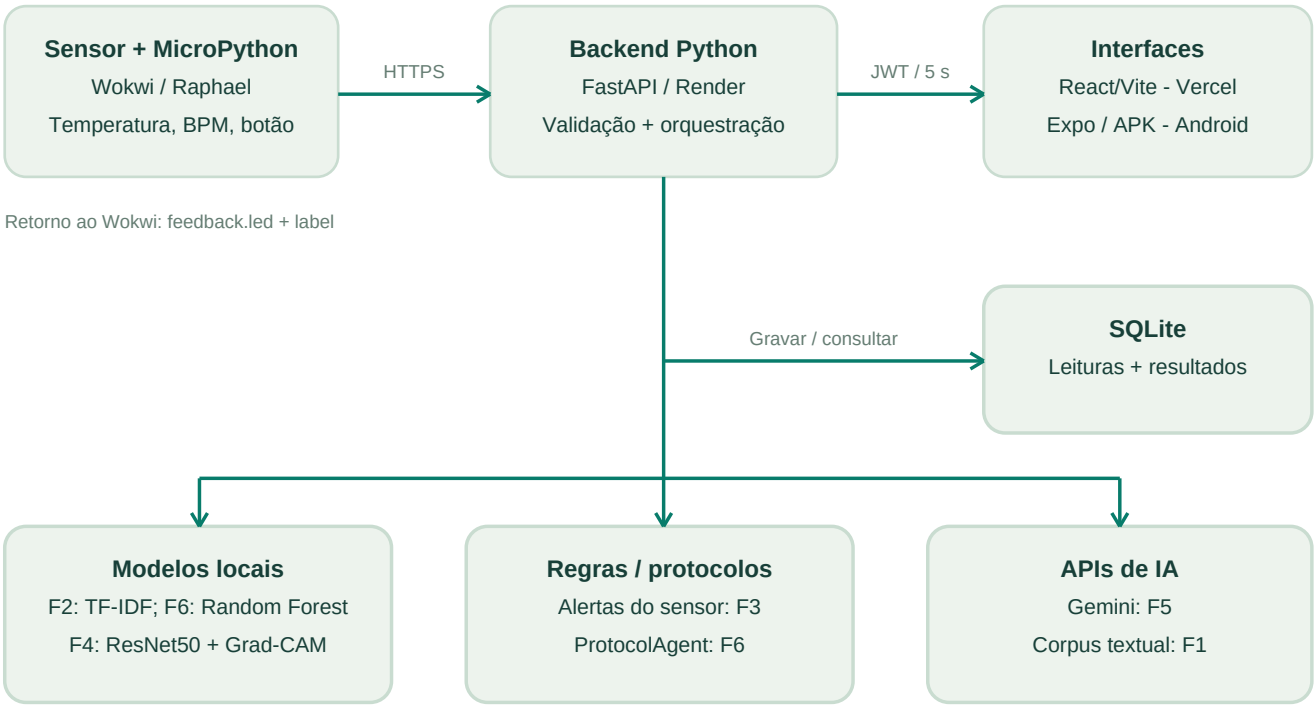
Fase	Componente consolidado
1	Corpus textual recuperado para contexto do assistente.
2	Extração de sintomas e TF-IDF + regressão logística; portal agora consome API real.
3	Regras de alertas preservadas; contrato HTTP autenticado para sensores.
4	ResNet50 treinada e Grad-CAM; fluxo de upload migrado para Web/Expo.
5	Assistência Gemini, mantendo credenciais somente no backend.
6	Random Forest original e ProtocolAgent, executados em sequência pelo Python.

Equipe e colaboração

Jonas Tadeu V. Fernandes (RM563027); Levi Passos Silveira Marques (RM56557); Raphael da Silva (RM561452); Raphael Dinelli Neto (RM562892); Yan Pimental Cotta (RM562836).

Yan: backend, interfaces, deploy e documentação. Raphael: MicroPython/Wokwi e feedback visual. Grupo: revisão, teste físico e vídeo. Tutor: Caique Nonato da Silva Bezerra. Coordenador: André Godoi.

Arquitetura e fluxo



Comunicação fim-a-fim

O dispositivo envia JSON a POST /api/v1/readings usando X-Device-Key. A API valida identidade, tipos, faixas e horário, calcula alertas e previsão, grava o evento e devolve a cor do LED. Web e APK consultam /dashboard a cada cinco segundos; após 30 segundos sem amostra recente, indicam dados antigos.

Cada leitura possui event_id. Reenvio idêntico é idempotente; repetir ID com conteúdo diferente retorna 409. O histórico é ordenado pelo horário da captura, impedindo que uma amostra atrasada substitua a mais recente.

Interpretação das previsões

O modelo F6 prevê risco **operacional hospitalar sintético**. BPM/temperatura vêm da simulação; os demais indicadores usam contexto sintético explícito, salvo SpO2/pressão quando fornecidos. A origem de cada feature é retornada pela API. Uma amostra de sensor não equivale à média de uma população: a integração é didática, sem validação clínica.

Os alertas e o LED seguem as regras F3 e são independentes da probabilidade F6. Protocolos F6 são apresentados como simulações. O Python executa os agentes de forma determinística; o chat Gemini explica a saída calculada, sem ser responsável por inventar valores ou executar ações.

Implementação e validação

Camada	Implementação
Backend	FastAPI/Python 3.12; JWT com 2 h; chave IoT restrita; SQLite; OpenAPI; limites de corpo/chamadas e CORS explícito.
Web	React + Vite. Login, painel, histórico, análise textual, imagem e chat. vercel.json com rotas SPA.
Mobile	React Native + Expo. Login, sinais, histórico, chat e imagem. android.package=br.com.cardioia.fiap; eas.json preview/APK.
IA	scikit-learn 1.9.1 compatível com joblib original. TensorFlow/Keras legado opcional para imagem. Gemini via HTTPS.
Entrega	Workflow GitHub Actions para testes e builds. Vercel publica após testes no push de main; Render usa checksPass. Fluxo publicado e validado.

Verificações executadas

27 testes de integração passaram: login, token expirado, acesso negado, chave/dispositivo, validação, idempotência, conflito, persistência, ordenação e dados antigos, modelo original, NLP, fallback/provedor, indisponibilidade de visão, limite de corpo e CORS.

O caso original da F6 retornou probabilidade 0,99, classe HIGH e protocolos P02/P05/P08/P10. Build Web, bundle Android e APK assinado no EAS concluíram. ResNet50 e Grad-CAM executaram em uma imagem sintética, verificando o runtime, sem medir acurácia.

Na Web pública: login e cenário de alerta atualizaram 132 bpm e 39,1 °C. O painel separou alerta do sensor HIGH e risco operacional LOW (18,5%). Gemini respondeu pela Web. Na API pública, 16 verificações passaram, incluindo sensores, modelos, chat e CORS.

O que ainda precisa ser comprovado

O APK foi gerado e seu arquivo verificado, mas a instalação em Android físico será validada por Raphael/grupo. Wokwi e vídeo fim-a-fim permanecem pendentes. O helper de transporte MicroPython não é uma simulação concluída. Evidências de deploy e testes estão em docs/evidence/.

Entrega e limitações

Hospedagem adequada ao MVP

O perfil gratuito Render publicou o núcleo leve, desativa a visão e usa disco efêmero. Pode suspender por inatividade; abrir /health antes da demonstração. A hospedagem de visão foi adiada por Yan. Para todos os módulos, preparar instância com pelo menos 2 GB, TensorFlow e disco persistente. O smoke local de visão atingiu cerca de 836 MB no macOS; isso não é medição Linux. Custos precisam ser aprovados antes da contratação.

Segurança e limites de uso

A aplicação tem um único usuário/paciente de demonstração e não deve receber dados reais de saúde. Tokens ficam em sessionStorage (Web) e SecureStore (mobile). Credenciais não são versionadas. Chave IoT só permite envio ao dispositivo/paciente configurado. O sistema não envia mensagens nem aciona serviços de emergência.

Modelos e base textual são educacionais. O classificador textual usa somente 50 exemplos; pode falhar com negação/contexto. O modelo F6 foi treinado em dados sintéticos e há diferença de distribuição. Os resultados não constituem diagnóstico nem indicação de tratamento.

Registro dos links

Entregável	Registro
Repositório	https://github.com/Phaeld/2TIAOA-CAP01-F7
Web Vercel	https://cardioia-fiap-final.vercel.app
Backend	https://cardioia-api-6efu.onrender.com
Expo / APK	Build EAS concluído: 01b49027-ce04-4310-8c76-30e3ada07c04. Link/QR e APK no README; teste físico pendente.
Wokwi / vídeo	Responsáveis: Raphael / grupo. Links pendentes.

Fechamento até 10/11/2026

Phaeld deve tornar o repositório privado e compartilhar com o tutor. Concluir Wokwi, instalar APK, coletar prints e gravar vídeo de até cinco minutos. Web, API, APK e CI/CD já possuem links/evidências no README. Fechar este relatório com Wokwi e teste físico, sem declarar conclusão antes da validação.

Referências: vercel.com/docs/project-configuration/vercel-json; docs.expo.dev/build-reference/apk; render.com/docs/compute-plans; render.com/docs/disks; ai.google.dev/api/generate-content; docs.wokwi.com/guides/esp32-wifi. Fontes, commits e checksums: docs/migration-manifest.json.